# 10/30/2026: Algorithm Efficiency

CSCI 246: Discrete Structures

Textbook reference: Sec 11.3, Epp

## Announcements

- The slides from Wednesday's class had two typos; they have been fixed.

## Today's Agenda

- Quiz #8
- Mini-lecture on algorithmic efficiency
- Group Exercises

# Weekly Quiz #8

1. *(Induction.)* Show by induction that $2^n < n!$ for all $n \geq 4$.
2. *(Recurrence.)* Solve the Tower of Hanoi recursion: $a_n = 2a_{n-1} + 1$ with $a_0 = 1$.
3. *(Recurrence.)* Show how to "outsmart" an intelligence test by finding a polynomial expression (of degree 3) for $a_n$ such that $a_0 = 1, a_1 = 2, a_2 = 4, a_3 = 8$, but $a_4 = 15$.

**Reference Material: Solving first-order recurrences (Props 23.1, 23.3)**

All solutions to recurrence relation $a_n = sa_{n-1} + t$ have the form

$$a_n = \begin{cases} c_1 s^n + c_2, & \text{if } s \neq 1 \\ a_0 + nt, & \text{if } s = 1 \end{cases}$$

**Reference Material: Expressing a sequence with a polynomial (Thm 23.17)**

Let $a_0, a_1, a_2, \ldots$ be a sequence of numbers. The terms $a_n$ can be expressed as polynomial expressions in $n$ if and only if there is a nonnegative integer $k$ such that for all $n \geq 0$ we have $\Delta^{k+1} a_n = 0$. In that case,

$$a_n = a_0 \binom{n}{0} + (\Delta a_0)\binom{n}{1} + (\Delta^2 a_0)\binom{n}{2} + \cdots + (\Delta^k a_0)\binom{n}{k}$$

# Mini-lecture

## Practice Reading Quiz (Algorithm Efficiency)

Assume $n$ is a positive integer and consider the following Python code:

```
1 p=0; x=2
2 for i in range(2, n + 1):          # i = 2, 3, ..., n
3     p=(p+i)*x
```

1. What is the actual number of elementary operations that are performed when this algorithm segment is executed? Justify your answer.
2. What is the order for this algorithm segment? Justify your answer.

## Solution to Practice Reading Quiz

```
1 p=0; x=2
2 for i in range(2, n + 1):          # i = 2, 3, ..., n
3     p=(p+i)*x
```

1. What is the actual number of elementary operations that are performed when this algorithm segment is executed?

## Solution to Practice Reading Quiz

```
1 p=0; x=2
2 for i in range(2, n + 1):          # i = 2, 3, ..., n
3     p=(p+i)*x
```

1. What is the actual number of elementary operations that are performed when this algorithm segment is executed? By the blue box below, $(n-1) \times 2 = 2n - 2$.

## The number of elementary operations performed by a for loop

The number of elementary operations performed by a for loop is given by

$$\#\text{elementary ops} = \#\text{iterations} \times \frac{\#\text{elementary ops}}{\text{iteration}}$$

## Solution to Practice Reading Quiz

```
1 p=0; x=2
2 for i in range(2, n + 1):          # i = 2, 3, ..., n
3     p=(p+i)*x
```

1. What is the actual number of elementary operations that are performed when this algorithm segment is executed? By the blue box below, $(n-1) \times 2 = 2n - 2$.

## The number of elementary operations performed by a for loop

The number of elementary operations performed by a for loop is given by

$$\#\text{elementary ops} = \#\text{iterations} \times \frac{\#\text{elementary ops}}{\text{iteration}}$$

## Solution to Practice Reading Quiz

```
1 p=0; x=2
2 for i in range(2, n + 1):          # i = 2, 3, ..., n
3     p=(p+i)*x
```

1. What is the actual number of elementary operations that are performed when this algorithm segment is executed? By the blue box below, $(n-1) \times 2 = 2n - 2$.
2. What is the order for this algorithm segment?

## The number of elementary operations performed by a for loop

The number of elementary operations performed by a for loop is given by

$$\#\text{elementary ops} = \#\text{iterations} \times \frac{\#\text{elementary ops}}{\text{iteration}}$$

## Solution to Practice Reading Quiz

```
1 p=0; x=2
2 for i in range(2, n + 1):          # i = 2, 3, ..., n
3     p=(p+i)*x
```

1. What is the actual number of elementary operations that are performed when this algorithm segment is executed? By the blue box below, $(n-1) \times 2 = 2n-2$.
2. What is the order for this algorithm segment? By the theorem on polynomial orders, $\Theta(n)$.

## The number of elementary operations performed by a for loop

The number of elementary operations performed by a for loop is given by

$$\#\text{elementary ops} = \#\text{iterations} \times \frac{\#\text{elementary ops}}{\text{iteration}}$$

# Nested for loops

**Poll.** Consider an algorithm segment with a nested for loop, such as the one below.

**An algorithm segment with a nested for loop**

```
1  for i in range(1, n + 1):          # i = 1, 2, ..., n
2      for j in range(1, i + 1):      # j = 1, 2, ..., i
3          print("YOLO.")
```

# Nested for loops

**Poll.** Consider an algorithm segment with a nested for loop, such as the one below.

**An algorithm segment with a nested for loop**

```
1  for i in range(1, n + 1):          # i = 1, 2, ..., n
2      for j in range(1, i + 1):      # j = 1, 2, ..., i
3          print("YOLO.")
```

How many times is the inner loop run?

# Nested for loops

**Poll.** Consider an algorithm segment with a nested for loop, such as the one below.

**An algorithm segment with a nested for loop**

```
1  for i in range(1, n + 1):          # i = 1, 2, ..., n
2      for j in range(1, i + 1):      # j = 1, 2, ..., i
3          print("YOLO.")
```

How many times is the inner loop run?

**Solution.** The number of times the inner loop is run is

$$1 + 2 + 3 + 4 + \ldots + n$$

# Nested for loops

**Poll.** Consider an algorithm segment with a nested for loop, such as the one below.

**An algorithm segment with a nested for loop**

```
1  for i in range(1, n + 1):          # i = 1, 2, ..., n
2      for j in range(1, i + 1):      # j = 1, 2, ..., i
3          print("YOLO.")
```

How many times is the inner loop run?

**Solution.** The number of times the inner loop is run is

$$1 + 2 + 3 + 4 + \ldots + n$$

**Why do we care?** We need this quantity to compute algorithmic efficiency:

**The number of elementary operations performed by a nested for loop**

The number of elementary operations performed by a nested for loop is given by

$$\#\text{elementary ops} = \#\ \underline{\text{inner}}\ \text{iterations} \times \frac{\#\text{elementary ops}}{\underline{\text{inner}}\ \text{iteration}}$$

## Theorem: Sum of the first $n$ natural numbers

For every natural number $n \geq 1$,

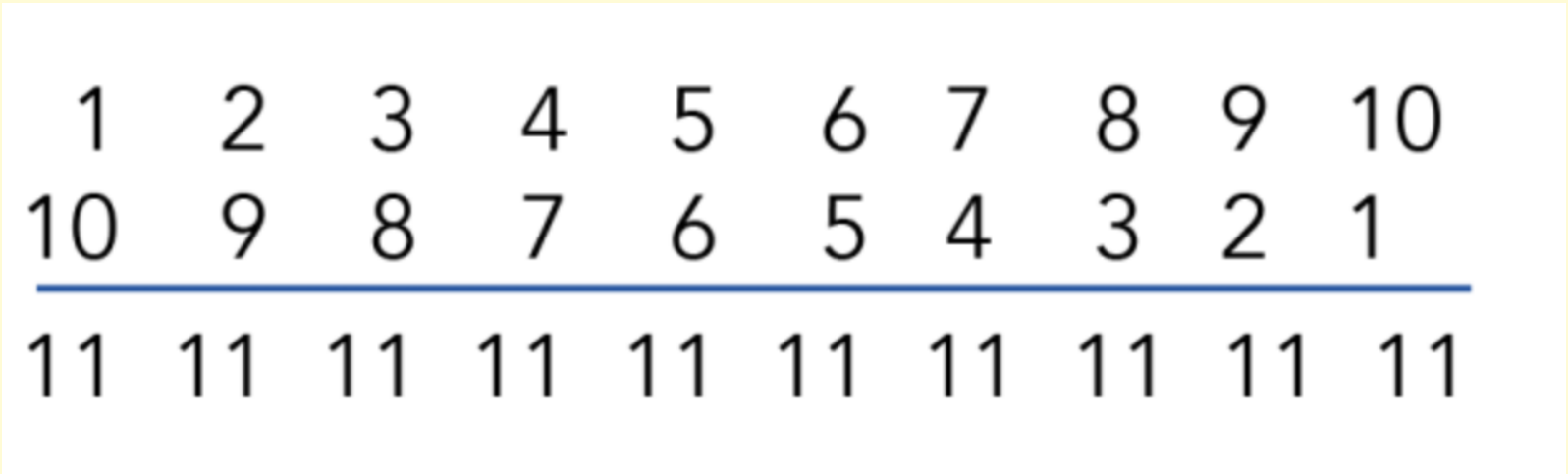
$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

## Theorem: Sum of the first $n$ natural numbers

For every natural number $n \geq 1$,

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

## Story: Gauss' solution from elementary school

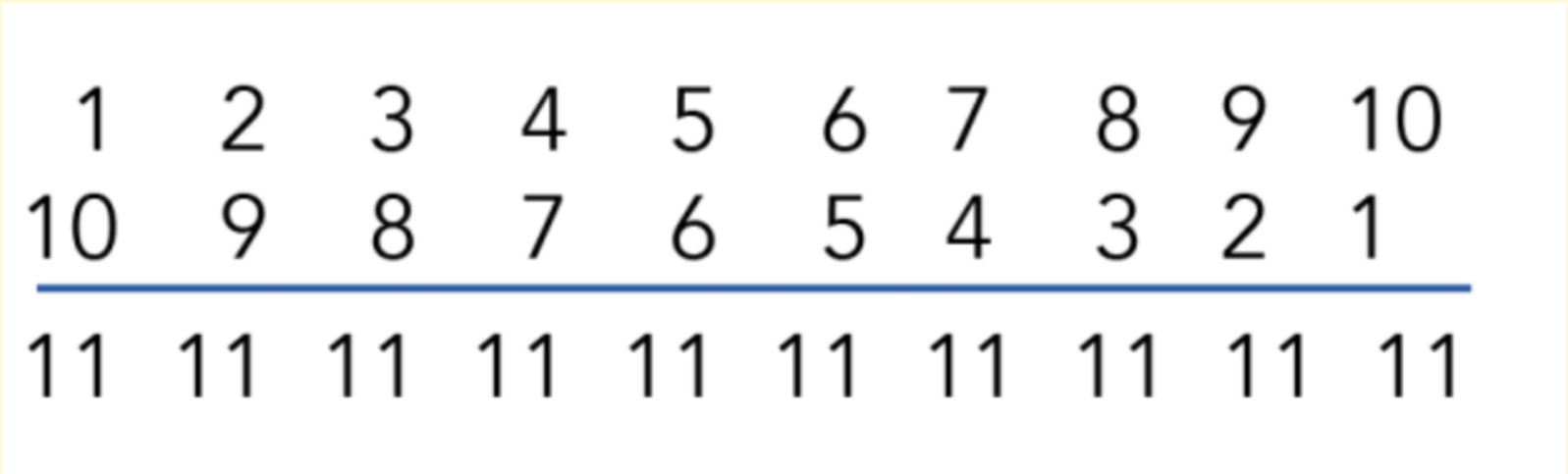


## Theorem: Sum of the first $n$ natural numbers

For every natural number $n \geq 1$,

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

## Story: Gauss' solution from elementary school




**Remark.** Recall that we can use summation notation to express the left-hand side more succinctly; in that case, we write the theorem as:

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

# Nested for loops

**Poll.** 1. What is the number of elementary operations performed by the code block below? 2. What is the order for this algorithm segment? Justify your answer.

**An algorithm segment (in Python) with a nested for loop**

```
1  x=0
2  for i in range(1, n + 1):          # i = 1, 2, ..., n
3      for j in range(1, i + 1):      # j = 1, 2, ..., i
4          x=x+i*j
```

# Nested for loops

**Poll.** 1. What is the number of elementary operations performed by the code block below? 2. What is the order for this algorithm segment? Justify your answer.

**An algorithm segment (in Python) with a nested for loop**

```
1  x=0
2  for i in range(1, n + 1):          # i = 1, 2, ..., n
3      for j in range(1, i + 1):      # j = 1, 2, ..., i
4          x=x+i*j
```

**The number of elementary operations performed by a nested for loop**

The number of elementary operations performed by a nested for loop is given by

$$\#\text{elementary ops} = \#\ \underline{\text{inner}}\ \text{iterations} \times \frac{\#\text{elementary ops}}{\underline{\text{inner}}\ \text{iteration}}$$

# Nested for loops

**Poll.** 1. What is the number of elementary operations performed by the code block below? 2. What is the order for this algorithm segment? Justify your answer.

**An algorithm segment (in Python) with a nested for loop**

```
1  x=0
2  for i in range(1, n + 1):          # i = 1, 2, ..., n
3      for j in range(1, i + 1):      # j = 1, 2, ..., i
4          x=x+i*j
```

**The number of elementary operations performed by a nested for loop**

The number of elementary operations performed by a nested for loop is given by

$$\#\text{elementary ops} = \#\ \underline{\text{inner}}\ \text{iterations} \times \frac{\#\text{elementary ops}}{\underline{\text{inner}}\ \text{iteration}}$$

**Solution to #1.**

$$(1 + 2 + 3 + \ldots + n) \cdot 2$$

# Nested for loops

**Poll.** 1. What is the number of elementary operations performed by the code block below? 2. What is the order for this algorithm segment? Justify your answer.

**An algorithm segment (in Python) with a nested for loop**

```
1  x=0
2  for i in range(1, n + 1):          # i = 1, 2, ..., n
3      for j in range(1, i + 1):      # j = 1, 2, ..., i
4          x=x+i*j
```

**The number of elementary operations performed by a nested for loop**

The number of elementary operations performed by a nested for loop is given by

$$\#\text{elementary ops} = \#\ \underline{\text{inner}}\ \text{iterations} \times \frac{\#\text{elementary ops}}{\underline{\text{inner}}\ \text{iteration}}$$

**Solution to #1.**

$$(1+2+3+\ldots+n)\cdot 2 = \frac{n(n+1)}{2}\cdot 2$$

# Nested for loops

**Poll.** 1. What is the number of elementary operations performed by the code block below? 2. What is the order for this algorithm segment? Justify your answer.

**An algorithm segment (in Python) with a nested for loop**

```
1  x=0
2  for i in range(1, n + 1):          # i = 1, 2, ..., n
3      for j in range(1, i + 1):      # j = 1, 2, ..., i
4          x=x+i*j
```

**The number of elementary operations performed by a nested for loop**

The number of elementary operations performed by a nested for loop is given by

$$\#\text{elementary ops} = \#\ \underline{\text{inner}}\ \text{iterations} \times \frac{\#\text{elementary ops}}{\underline{\text{inner}}\ \text{iteration}}$$

**Solution to #1.**

$$(1+2+3+\ldots+n)\cdot 2 = \frac{n(n+1)}{2}\cdot 2 = n(n+1)$$

# Nested for loops

**Poll.** 1. What is the number of elementary operations performed by the code block below? 2. What is the order for this algorithm segment? Justify your answer.

**An algorithm segment (in Python) with a nested for loop**

```
1  x=0
2  for i in range(1, n + 1):          # i = 1, 2, ..., n
3      for j in range(1, i + 1):      # j = 1, 2, ..., i
4          x=x+i*j
```

**The number of elementary operations performed by a nested for loop**

The number of elementary operations performed by a nested for loop is given by

$$\#\text{elementary ops} = \#\ \underline{\text{inner}}\ \text{iterations} \times \frac{\#\text{elementary ops}}{\underline{\text{inner}}\ \text{iteration}}$$

**Solution to #1.**

$$(1+2+3+\ldots+n)\cdot 2 = \frac{n(n+1)}{2}\cdot 2 = n(n+1) = n^2+n$$

# Nested for loops

**Poll.** 1. What is the number of elementary operations performed by the code block below? 2. What is the order for this algorithm segment? Justify your answer.

**An algorithm segment (in Python) with a nested for loop**

```
1  x=0
2  for i in range(1, n + 1):          # i = 1, 2, ..., n
3      for j in range(1, i + 1):      # j = 1, 2, ..., i
4          x=x+i*j
```

**The number of elementary operations performed by a nested for loop**

The number of elementary operations performed by a nested for loop is given by

$$\#\text{elementary ops} = \#\ \underline{\text{inner}}\ \text{iterations} \times \frac{\#\text{elementary ops}}{\underline{\text{inner}}\ \text{iteration}}$$

**Solution to #1.**

$$(1+2+3+\ldots+n)\cdot 2 = \frac{n(n+1)}{2}\cdot 2 = n(n+1) = n^2+n$$

# Nested for loops

**Poll.** 1. What is the number of elementary operations performed by the code block below? 2. What is the order for this algorithm segment? Justify your answer.

**An algorithm segment (in Python) with a nested for loop**

```
1  x=0
2  for i in range(1, n + 1):          # i = 1, 2, ..., n
3      for j in range(1, i + 1):      # j = 1, 2, ..., i
4          x=x+i*j
```

**The number of elementary operations performed by a nested for loop**

The number of elementary operations performed by a nested for loop is given by

$$\#\text{elementary ops} = \#\ \underline{\text{inner}}\ \text{iterations} \times \frac{\#\text{elementary ops}}{\underline{\text{inner}}\ \text{iteration}}$$

**Solution to #1.**

$$(1+2+3+\ldots+n)\cdot 2 = \frac{n(n+1)}{2}\cdot 2 = n(n+1) = n^2+n$$

# Remark: Big O is not "worst case" complexity

| **Method for computing # elem. operations** | | **Method for expressing order of algorithm** |
|---|---|---|
| Worst Case | $\times$ | Big O |
| Best Case | | Big Theta |
| Average Case | | Big Omega |

# Big O is not "worst case" complexity – Example

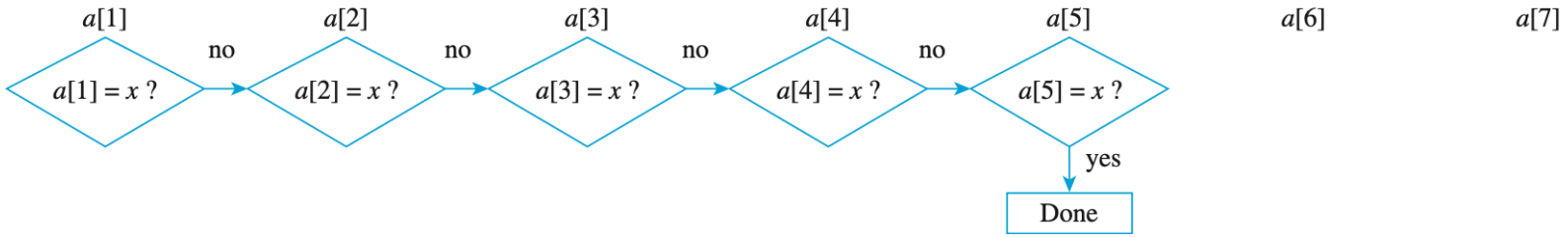



**FIGURE 11.3.1** Sequential Search of $a[1], a[2], \ldots, a[7]$ for $x$ where $x = a[5]$

# Big O is not "worst case" complexity – Example

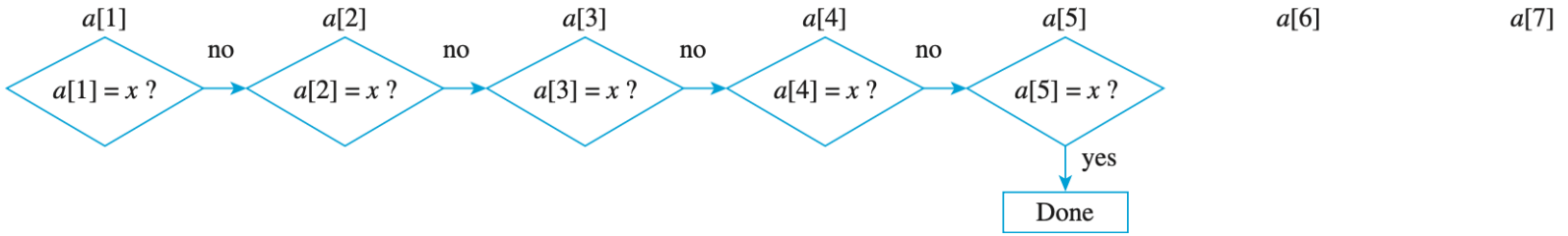



**FIGURE 11.3.1** Sequential Search of $a[1], a[2], \ldots, a[7]$ for $x$ where $x = a[5]$

**Poll.** How many elementary operations are required by sequential search over $n$ items in (a) the best case scenario, (b) the worst case scenario, (c) the average case scenario?

# Big O is not "worst case" complexity – Example

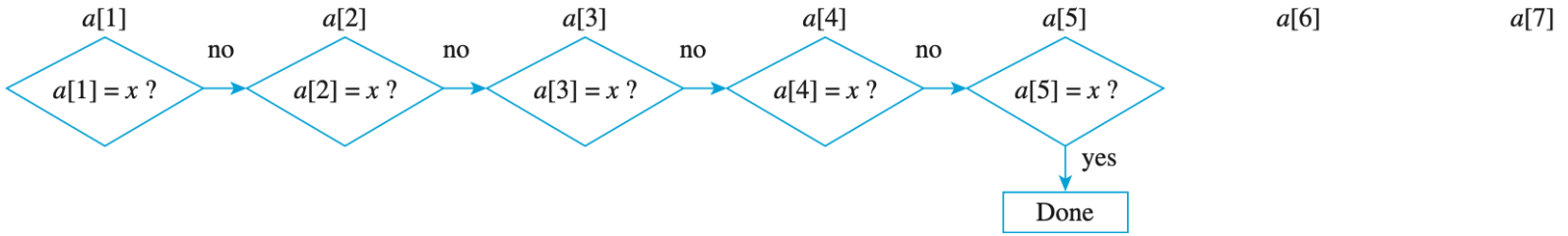



**FIGURE 11.3.1** Sequential Search of $a[1], a[2], \ldots, a[7]$ for $x$ where $x = a[5]$

**Poll.** How many elementary operations are required by sequential search over $n$ items in (a) the best case scenario, (b) the worst case scenario, (c) the average case scenario?

**Solution.** (a) 1 comparison

# Big O is not "worst case" complexity – Example

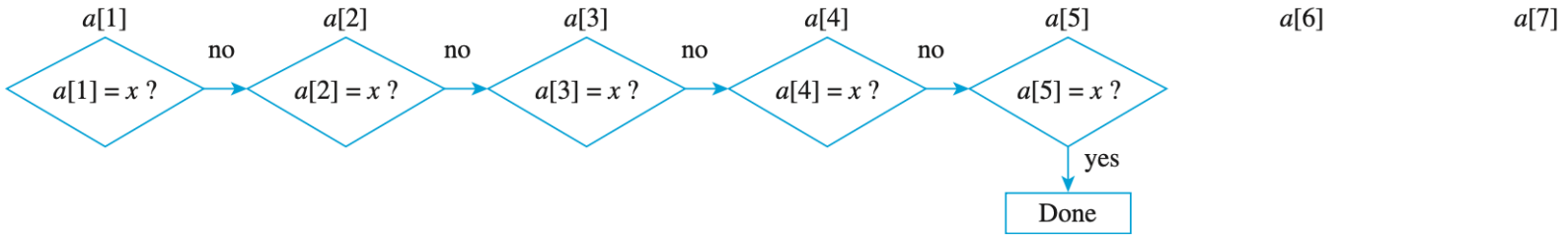



**FIGURE 11.3.1** Sequential Search of $a[1], a[2], \ldots, a[7]$ for $x$ where $x = a[5]$

**Poll.** How many elementary operations are required by sequential search over $n$ items in (a) the best case scenario, (b) the worst case scenario, (c) the average case scenario?

**Solution.** (a) 1 comparison (b) $n$ comparisons

# Big O is not "worst case" complexity – Example

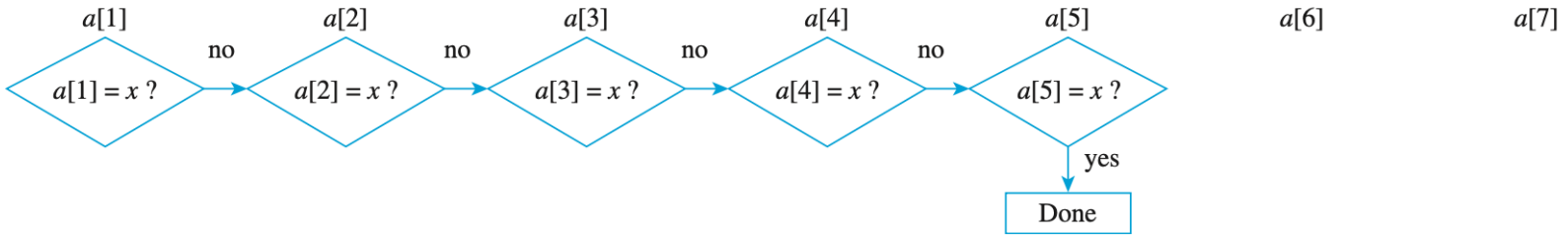



**FIGURE 11.3.1** Sequential Search of $a[1], a[2], \ldots, a[7]$ for $x$ where $x = a[5]$

**Poll.** How many elementary operations are required by sequential search over $n$ items in (a) the best case scenario, (b) the worst case scenario, (c) the average case scenario?

**Solution.** (a) 1 comparison (b) $n$ comparisons (c) $\frac{n+1}{2}$ comparisons (if the item of interest is equally likely to be anywhere in the array).

# Big O is not "worst case" complexity – Example

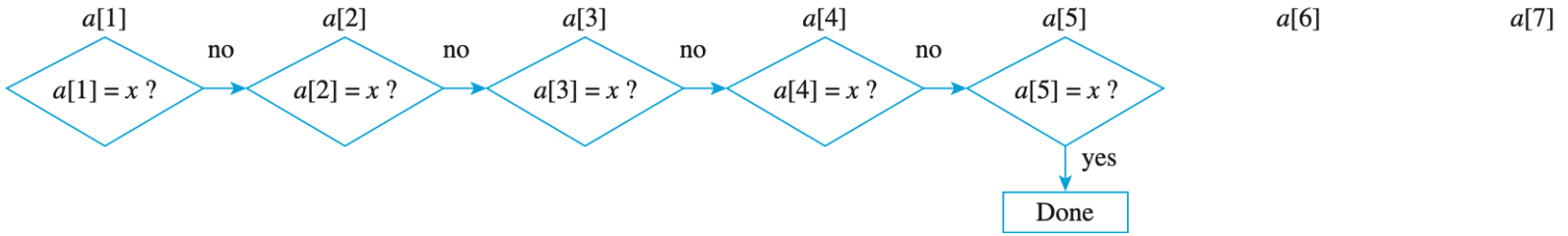



**FIGURE 11.3.1** Sequential Search of $a[1], a[2], \ldots, a[7]$ for $x$ where $x = a[5]$

**Poll.** How many elementary operations are required by sequential search over $n$ items in (a) the best case scenario, (b) the worst case scenario, (c) the average case scenario?

**Solution.** (a) 1 comparison (b) $n$ comparisons (c) $\frac{n+1}{2}$ comparisons (if the item of interest is equally likely to be anywhere in the array).

**Remark.** Hence, the order in each case is (a) $\Theta(1)$, (b) $\Theta(n)$, (c) $\Theta(n)$.

# Big O is not "worst case" complexity – Example

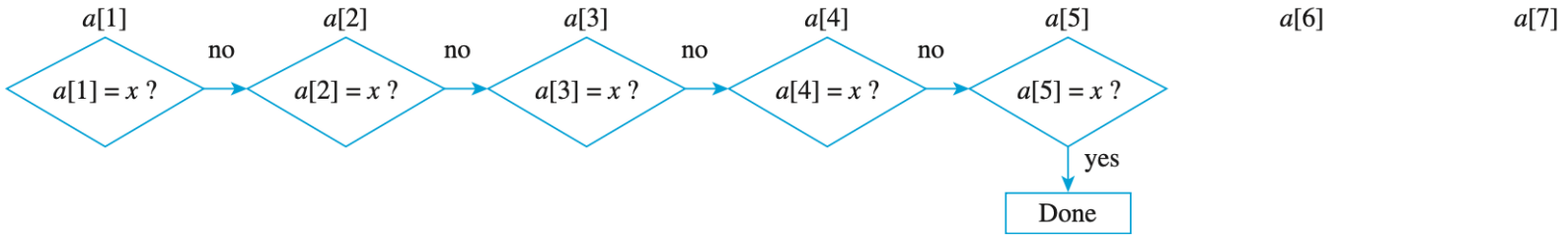



**FIGURE 11.3.1** Sequential Search of $a[1], a[2], \ldots, a[7]$ for $x$ where $x = a[5]$

**Poll.** How many elementary operations are required by sequential search over $n$ items in (a) the best case scenario, (b) the worst case scenario, (c) the average case scenario?

**Solution.** (a) 1 comparison (b) $n$ comparisons (c) $\frac{n+1}{2}$ comparisons (if the item of interest is equally likely to be anywhere in the array).

**Remark.** Hence, the order in each case is (a) $\Theta(1)$, (b) $\Theta(n)$, (c) $\Theta(n)$.

Thus, an upper bound on the order for each case is (a) $O(1)$, (b) $O(n)$, (c) $O(n)$. Likewise, a lower bound on the order for each case is (a) $\Omega(1)$, (b) $\Omega(n)$, (c) $\Omega(n)$.

# Two separate questions

**Question 1. Which inputs?** Choosing a **case** gives a **function** of $n$:

- $W(n)$ = number of operations on the **worst** input of size $n$
- $B(n)$ = number of operations on the **best** input of size $n$
- $A(n)$ = **average** number of operations over inputs of size $n$

# Two separate questions

**Question 1. Which inputs?** Choosing a **case** gives a **function** of $n$:

- $W(n)$ = number of operations on the **worst** input of size $n$
- $B(n)$ = number of operations on the **best** input of size $n$
- $A(n)$ = **average** number of operations over inputs of size $n$

**Question 2. How do we describe that function?**

- $O(g)$: grows **at most** like $g$ (upper bound)
- $\Omega(g)$: grows **at least** like $g$ (lower bound)
- $\Theta(g)$: grows **exactly** like $g$ (both)

# Two separate questions

**Question 1. Which inputs?** Choosing a **case** gives a **function** of $n$:

- $W(n)$ = number of operations on the **worst** input of size $n$
- $B(n)$ = number of operations on the **best** input of size $n$
- $A(n)$ = **average** number of operations over inputs of size $n$

**Question 2. How do we describe that function?**

- $O(g)$: grows **at most** like $g$ (upper bound)
- $\Omega(g)$: grows **at least** like $g$ (lower bound)
- $\Theta(g)$: grows **exactly** like $g$ (both)

**Take-away**

Cases (worst/best/average) pick *which function* to study. $O$, $\Omega$, $\Theta$ describe *how fast it grows*. Any case can be paired with any symbol.

# Example: sequential search over $n$ items

| | # comparisons | $O$ | $\Omega$ | $\Theta$ |
|---|---|---|---|---|
| **Best case** | 1 | $O(1)$ | $\Omega(1)$ | $\Theta(1)$ |
| **Worst case** | $n$ | $O(n)$ | $\Omega(n)$ | $\Theta(n)$ |
| **Average case** | $\frac{n+1}{2}$ | $O(n)$ | $\Omega(n)$ | $\Theta(n)$ |

# Example: sequential search over $n$ items

| | # comparisons | $O$ | $\Omega$ | $\Theta$ |
|---|---|---|---|---|
| **Best case** | 1 | $O(1)$ | $\Omega(1)$ | $\Theta(1)$ |
| **Worst case** | $n$ | $O(n)$ | $\Omega(n)$ | $\Theta(n)$ |
| **Average case** | $\frac{n+1}{2}$ | $O(n)$ | $\Omega(n)$ | $\Theta(n)$ |

**Note.** All nine entries are true statements. For example, "the worst case is $\Omega(n)$" is a perfectly good *lower* bound on the *worst* case.

# Big O is not "worst case" complexity – Summary

You'll hear "Big O means worst case." It's a common shorthand because people usually bound the worst case from above, but O is really an upper bound, and you can upper-bound any case.

Theoretical computer scientists care just as much about lower bounds on the worst case, like $\Omega(n \log n)$ for sorting.

# Group exercises

# Random group assignments

Ashley, Isabelle: 9
Bailey, Tanna: 9
Blaisdell, Olyver: 4
Brown, Ashton: 1
Cardenas, Dominic: 10
Cutler, George: 1
Fabik, Roland: 7
Fellows, Audrey: 6
Fitzgerald, Ryan: 6
Fuller, Ryan: 3
Griffen, Clara: 3
Grutsch, Tanner: 5
Hager, Nathan: 7
Horner, Jayden: 8
Johnson, Jaret: 2
Kintzel, Samantha: 12
Lameres, Alexis: 5
Lee, Holden: 4
Mayala, Reece: 10
McLean, Connor: 2
Mcbride, Daniel: 4
Miller, Maverick: 7
Moss, Emily: 10
Mousseau, Matthew: 12
Neuman, Evan: 11
Putnam, John: 8
Ramos, Vydel: 5
Rathbun, Jedidiah: 11
Roberts, Matthew: 6
Roller, Nathan: 2
Ryan, Otis: 1
Shevtsov, Timothy: 11
Stensland, Emma: 12
Thiessen, Logan: 8
Thompson, Jaiden: 3
Vath, Ethan: 9

# Group exercises

For each of the algorithm segments below, assume $n$ is a positive integer.

a. Compute the actual number of elementary operations (additions, subtractions, multiplications, divisions, and comparisons) that are performed when the algorithm segment is executed. For simplicity, however, count only comparisons that occur within the body of the for loops; ignore those required to determine when the for loops should terminate.

b. Use the theorem on polynomial orders to find an order for the algorithm segment.

Segment 1

```
for i in range(3, n):
    a = 3*n + 2*i - 1
```

Segment 2

```
for k in range(1,n):
    for j in range(1, k):
        x = a[k] + b[j]
```

Segment 3

```
for i in range(1,n+1):
    m = np.floor((i+1)/2)
    for j in range(1,m+1):
        a=(n-i)*(n-j)
```

# Solution for algorithm segment #1

```
for i in range(3, n):
    a = 3*n + 2*i - 1
```

**Solution Summary.**

| | | |
|---|---|---|
| Step 1 | **# its.** | $n-3$ |
| Step 2 | **# el ops. per it.** | 4 |
| Step 3 | **# el ops.** | $4n-12$ |
| Step 4 | **Order** | $\Theta(n)$ |

**Solution Details.**

Step 1 Since we skip the first two iterates, do $(n-1)-2 = n-3$.

Step 2 Two multiplications, one subtraction, one addition.

Step 3 Multiply Steps 1 and 2.

Step 4 By theorem on polynomial orders.

# Solution for algorithm segment #2

```
for k in range(1,n):
    for j in range(1, k):
        x = a[k] + b[j]
```

**Solution Summary.**

| | | |
|---|---|---|
| Step 1 | **# its.** | $\frac{n^2}{2} + \frac{n}{2} - 1$ |
| Step 2 | **# el ops. per it.** | 1 |
| Step 3 | **# el ops.** | $\frac{n^2}{2} + \frac{n}{2} - 1$ |
| Step 4 | **Order** | $\Theta(n^2)$ |

**Solution Details.**

Step 1 We unfold the nested loop into a single loop by focusing only on the inner loop. The inner loop does $2 + \ldots + n = \frac{n(n+1)}{2} - 1$ iterations. Here we use Gauss' formula for summing the first $n$ natural numbers.

Step 2 One addition.

Step 3 Multiply Steps 1 and 2.

Step 4 By theorem on polynomial orders.

# Solution for algorithm segment #3

**Solution Summary.**

```
for i in range(1,n+1):
    m = np.floor((i+1)/2)
    for j in range(1,m+1):
        a=(n-i)*(n-j)
```

| | | |
|---|---|---|
| Step 1 | **# its.** | $\begin{cases} \frac{n^2}{4}+\frac{n}{2}, & n \text{ even} \\ \frac{n^2}{4}+\frac{n}{2}+\frac{1}{4}, & n \text{ odd} \end{cases}$ |
| Step 2 | **# el ops. per it.** | 3 |
| Step 3 | **# el ops.** | $\begin{cases} 3\left(\frac{n^2}{4}+\frac{n}{2}\right), & n \text{ even} \\ 3\left(\frac{n^2}{4}+\frac{n}{2}+\frac{1}{4}\right), & n \text{ odd} \end{cases}$ |
| Step 4 | **Order** | $\Theta(n^2)$ |

**Solution Details.** How many times is the inner loop run? Depends if $n$ is odd or even.

$$\texttt{\# its } (n \texttt{ even}) = 1+1+2+2+\ldots+\frac{n}{2}+\frac{n}{2}$$

$$= 2\sum_{k=1}^{\frac{n}{2}} k \overset{*}{=} 2\frac{1}{2}(\frac{n}{2})(\frac{n+2}{2}) = \frac{n^2}{4}+\frac{n}{2}$$

$$\texttt{\# its } (n \texttt{ odd}) = 1+1+2+2+\ldots+\frac{n-1}{2}+\frac{n-1}{2}+\frac{n+1}{2}$$

$$= 2\sum_{k=1}^{\frac{n-1}{2}} k + \frac{n+1}{2} \overset{*}{=} 2\frac{1}{2}(\frac{n-1}{2})(\frac{n+1}{2}) + \frac{n+1}{2} = \frac{n^2}{4}+\frac{n}{2}+\frac{1}{4}$$

In the equations marked *, we use Gauss' formula for summing the first $n$ natural numbers.